Unit 1
Computer Systems

Digital logic · LogiSim · SDLC · OSI & TCP/IP

BOOLEAN ALGEBRA | LOGIC GATES | K-MAPS | WATERFALL VS AGILE | 7-LAYER OSI

How to use this booklet. Read one section, redraw every SVG from memory, then attempt the self-check before turning the page. Formulas are written in MathJax so you see the same notation an examiner expects.

0. What this unit is really about

Three stories share one chapter. First, computers think in only two voltages — ON and OFF — and Boolean algebra is the mathematics of those two states. Second, software is not "typed until it works"; it is built through a life cycle. Third, networks do not shout raw bits at each other; they wrap messages in layers. If you can move fluently among bits, gates, sprints, and packets, you understand how a modern computer system lives.

WHY IT MATTERS

Every later unit — Python, databases, IoT, HCI — sits on this foundation. A logic error in a gate, a skipped test in Waterfall, or a missing transport checksum all produce the same result: a system that looks finished and then fails.

YOU WILL BE ABLE TO

- Separate analog from digital, continuous from discrete
- Write Boolean expressions and truth tables
- Draw and simplify logic circuits (including K-maps)
- Compare Waterfall and Agile with the Al-Noor cases
- Map OSI's seven layers onto TCP/IP's four

EXAM HABITS

- Always state $0 = \text{LOW/FALSE}$ and $1 = \text{HIGH/TRUE}$
- Precedence: parentheses $\rightarrow$ NOT $\rightarrow$ AND $\rightarrow$ OR
- K-map columns use Gray code: $00, 01, 11, 10$
- Name the data unit at each layer (segment, packet, frame, bits)

1. Discrete vs continuous, analog vs digital

A **discrete** quantity takes separate countable values (students in a room, books on a shelf). A **continuous** quantity can sit anywhere in a range (temperature, height, speed). The stairs-versus-ramp picture is the whole idea: stairs force you onto fixed treads; a ramp lets you stop at $1.73\,\text{m}$ just as easily as at $1.5\,\text{m}$.

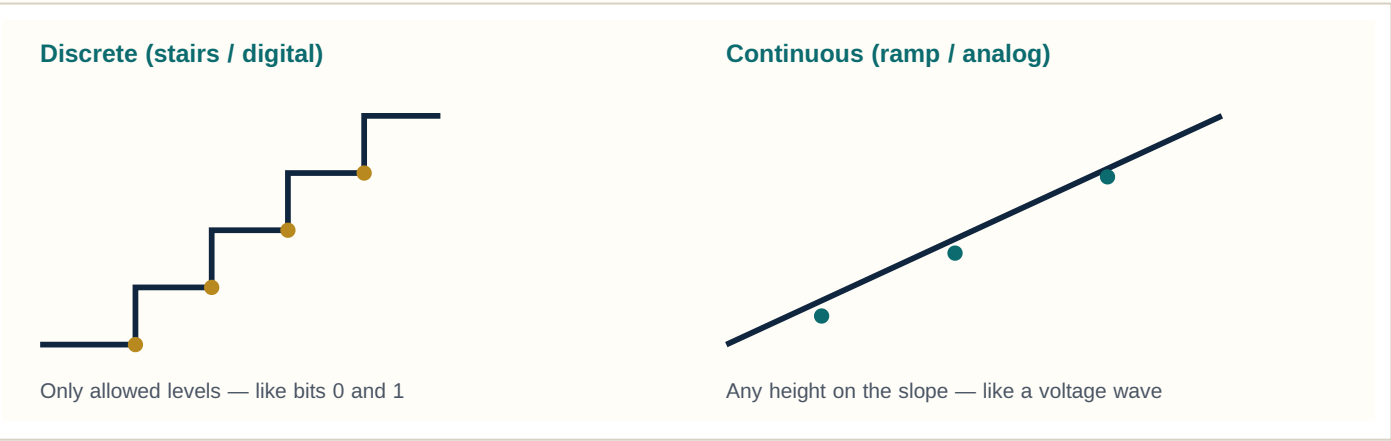


Figure 1. Stairs force discrete steps; a ramp is continuous. Digital systems choose the stairs.

A **digital system** uses only two values, called **bits**:

$$0 = \text{OFF} \;/\; \text{LOW} \;/\; \text{FALSE}, \qquad 1 = \text{ON} \;/\; \text{HIGH} \;/\; \text{TRUE}$$

Why industry chose this poverty of values: noise rarely flips a solid HIGH into a solid LOW; copies stay exact; designers handle two states, not infinite voltages; software can reprogram the same hardware.

Analog signal

A **signal** is a function that carries information. Analog signals vary smoothly over a continuous range — voice, guitar strings, mercury thermometers, old telephone voltages. Their graph is a sine-like wave. They are easily damaged by noise.

Digital signal

Digital signals jump between HIGH and LOW, drawing a square wave. Computer buses, USB, HDMI, and internet packets are digital. Reliability is high because a slightly dented pulse is still recognized as 1 or 0.

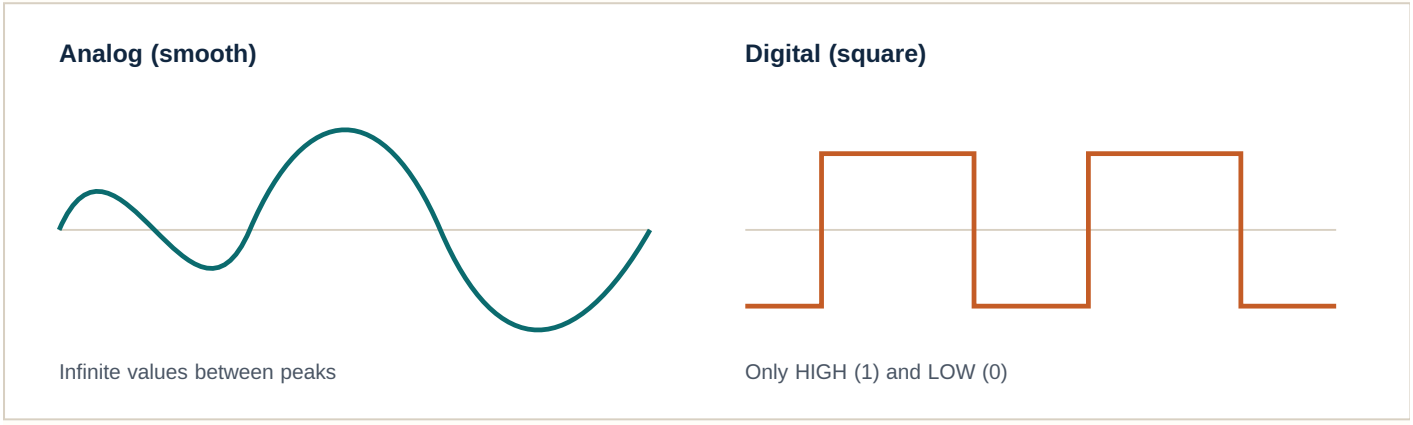


Figure 2. Wave shapes you must be able to sketch in an exam.

Table 1. Analog versus digital at a glance

Feature	Analog	Digital
Nature	Continuous	Discrete
Unit	Physical quantity (voltage, frequency)	Bit (0 or 1)
Wave	Smooth / sine	Square
Values	Infinite in a range	Two levels
Noise	Easily corrupted	Resistant
Examples	Voice, vinyl, mercury thermometer	PC, smartphone, digital watch

2. Boolean algebra — the algebra of true and false

George Boole (1850s) built a mathematics with only two constants. A **variable** such as A, B, C holds 0 or 1. An **expression** combines variables with AND, OR, and NOT. This is the language of every logic gate and of every `if` condition you will later write in Python.

AND · CONJUNCTION

$Y = A \cdot B$ or $Y = A \wedge B$

Output is 1 only when *all* inputs are 1. Door with two locks: both must open.

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

OR · DISJUNCTION

$Y = A + B$ or $Y = A \vee B$

Output is 1 when *any* input is 1. Room with two doors: either door lets you in.

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

NOT · COMPLEMENT (UNARY)

$Y = A'$ or $Y = \overline{A}$

Inverts one input. Light switch: ON ↔ dark? No — think of a *normally-on* lamp that is dark when the switch bit is 1.

A	$\overline{A}$
0	1
1	0

WATCH

In Boolean algebra the symbol $+$ is OR, not ordinary plus. $1 + 1 = 1$, never 2. The product $A \cdot B$ is AND. If a question writes $Y = A + B \cdot C$, AND happens first, so $Y = A + (B \cdot C)$.

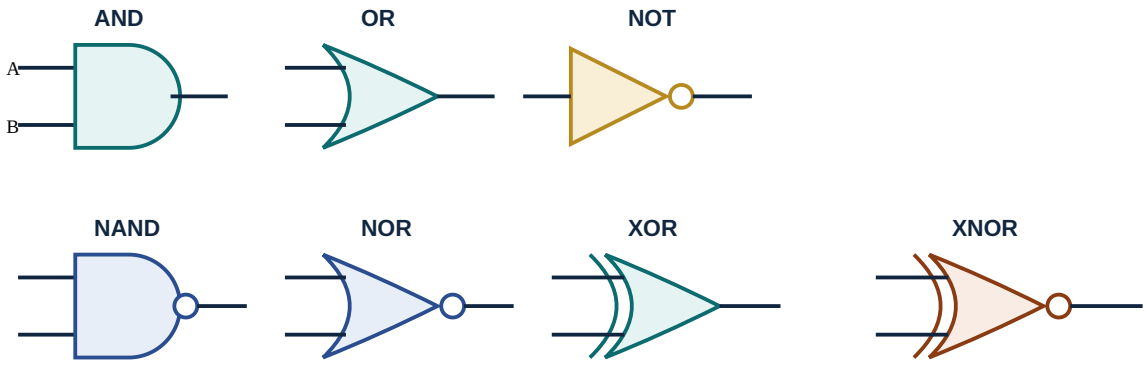
2.1 Truth tables from n inputs

A table with n input variables has exactly 2^n rows. List inputs in binary counting order so you never skip a combination.

$$\text{rows} = 2^n \qquad (n = 2 \Rightarrow 4,\ n = 3 \Rightarrow 8,\ n = 4 \Rightarrow 16)$$

3. Logic gates — hardware that obeys Boolean laws

A gate is a circuit with one or more inputs and one output. Learn the shape, the expression, and the English rule together.



Bubble on the output = inversion. Extra curve on XOR/XNOR = “exclusive”.
NAND and NOR are universal: any circuit can be built from only NAND, or only NOR.

Figure 3. Standard IEEE/ANSI gate outlines. Practise these until you can draw them in 30 seconds.

Table 2. Gate identities you must memorise

Gate	Expression	$Y = 1$ when	Everyday picture
AND	$Y = A \cdot B$	All inputs 1	Car starts: key AND start button
OR	$Y = A + B$	Any input 1	Alarm: front OR back door
NOT	$Y = \overline{A}$	Input is 0	Inverter / opposite lamp
NAND	$Y = \overline{A \cdot B}$	Any input 0	Universal gate
NOR	$Y = \overline{A + B}$	All inputs 0	Universal gate
XOR	$Y = A \oplus B = \overline{A}B + A\overline{B}$	Inputs differ	Two-way staircase light
XNOR	$Y = \overline{A \oplus B} = AB + \overline{A}\,\overline{B}$	Inputs same	Equality detector

3.1 Worked circuit: $Y = A \cdot B + C$

Precedence says AND first. Draw an AND of A and B, then OR that result with C. For three variables the truth table has 8 rows. The output is 1 whenever the product AB is 1, or C is 1.

$$Y = (A \cdot B) + C$$

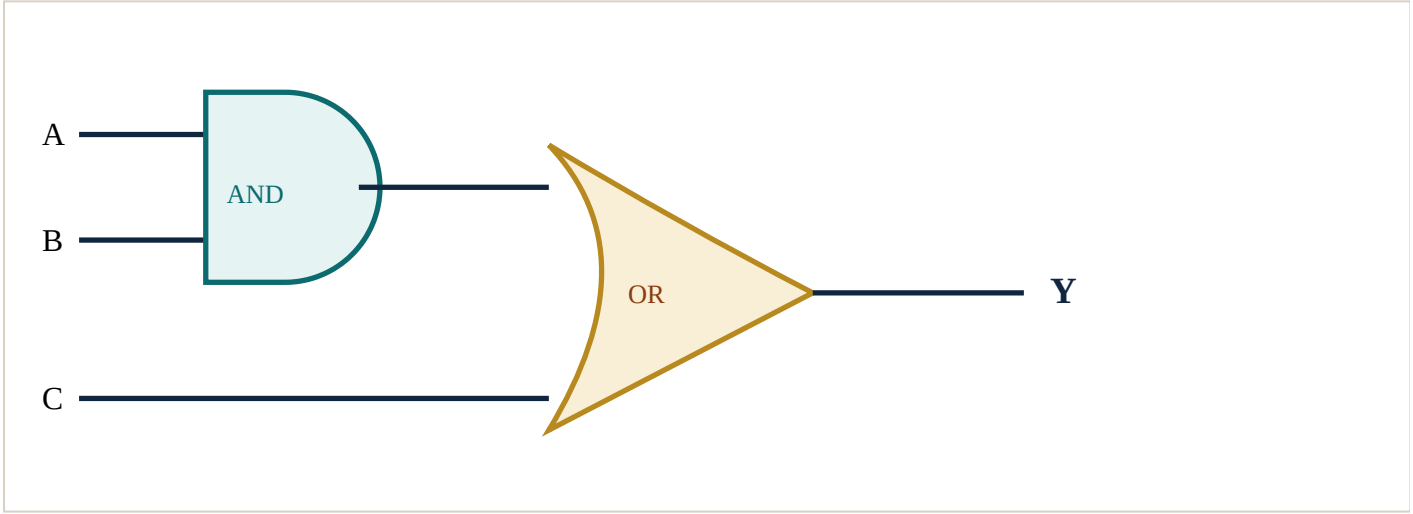


Figure 4. Logic diagram for $Y = A \cdot B + C$. Always AND the product terms, then OR them (SOP style).

3.2 Canonical forms — minterms and maxterms

A **minterm** is an AND of every variable (complemented if that variable is 0 in the row). You write one minterm for each row where $Y = 1$, then OR them. That is the **sum of products (SOP)**.

A **maxterm** is an OR of every variable (complemented if that variable is 1 in the row). You write one maxterm for each row where $Y = 0$, then AND them. That is the **product of sums (POS)**.

For two variables:

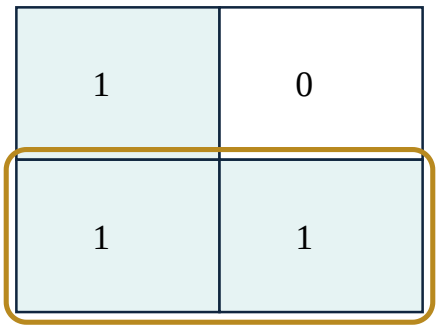
$$\begin{aligned} m_0 &= \overline{A}\,\overline{B}, & m_1 &= \overline{A}B, & m_2 &= A\overline{B}, & m_3 &= AB \\ M_0 &= A + B, & M_1 &= A + \overline{B}, & M_2 &= \overline{A} + B, & M_3 &= \overline{A} + \overline{B} \end{aligned}$$

If $Y = 1$ only on rows $(0,1)$ and $(1,1)$, then SOP is $Y = \overline{A}B + AB$, which simplifies to $Y = B$.

4. Karnaugh maps — seeing simplification

Maurice Karnaugh (1953) arranged truth-table 1s so that neighbouring cells differ by one bit (**Gray code**). You circle groups whose size is a power of two: $1, 2, 4, 8, \ldots$ Wrap around the edges; overlapping is allowed; bigger groups give simpler expressions.

2-variable map



Group the three 1s as A (bottom row) plus B wait — here the pair of bottom 1s and the left column give $Y = A + \overline{B}$ depending on cells. Always read which variable stays constant inside a circle.

Worked 3-variable collapse

Given

$$Y = \overline{A}\,\overline{B}C + \overline{A}BC + A\overline{B}C + ABC$$

every minterm contains C and the pairs cover both A and B. The whole map of 1s is the single group "C is 1":

$$Y = C$$

Gray-code column order for three variables is $BC = 00, 01, 11, 10$ — never $00, 01, 10, 11$. Adjacent cells, including the wrap from last column to first, differ by one bit.

Self-check 1 — Boolean fluency

1. Write the truth table of $Y = \overline{A} + B$.
2. Why is NAND called universal?
3. Simplify $Y = A\overline{B} + AB$ by inspection (no K-map required).

Answers. (1) $Y = 0$ only when $A = 1, B = 0$; otherwise 1. (2) Any Boolean function can be built using only NAND gates (NOT $= \mathrm{NAND}(A, A)$; AND $= \overline{\mathrm{NAND}}$; OR via De Morgan). (3) $Y = A(\overline{B} + B) = A \cdot 1 = A$.

5. LogiSim Evolution — from paper to a live circuit

LogiSim Evolution is the course simulation tool. You need Java 21+ (Adoptium) and the Windows `.msi` from the project's GitHub releases. Files use the `.circ` extension.

INTERFACE
Explorer (gate library) · canvas · Attribute Table · Simulate / Poke tool

WIRING RULE
Wires are horizontal or vertical only. A green circle appears when a pin will accept a connection.

COLOURS
Bright green $= 1$, dark green $= 0$. Input pins are squares you toggle; output pins are circles.

1. File → New, save as `AND_gate.circ`.
2. Place an AND gate, two input pins (A, B) on the left, one output pin (Y) on the right.
3. Wire pins to the gate. Use the Poke tool to toggle A and B and confirm $Y = A \cdot B$.

4 Extension lab: build $Y = A \cdot B + C$ (Figure 4) and tick all eight input combinations against your truth table.

6. Software Development Life Cycle (SDLC)

Building software is like building a house: you do not pour concrete and then ask how many bedrooms. The SDLC is the agreed sequence of phases, each with a deliverable. It gives clear goals, planned work, quality checks, risk control, documentation, and a shared language for the team.

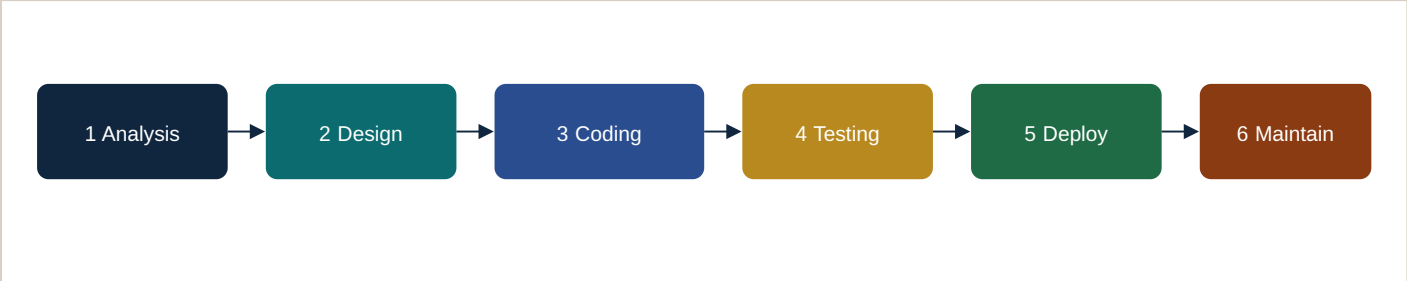


Figure 5. Six SDLC phases. In Waterfall the arrow is one-way; in Agile you repeat a short version of this arrow every sprint.

Table 3. What each phase actually produces

Phase	You do	Typical deliverable
Requirement analysis	Interview users, freeze needs	SRS (Software Requirements Specification)
System design	Architecture, database, screens	HLD / LLD, ER diagrams, wireframes
Implementation	Write code, use Git, review	Source code
Testing	Unit, integration, system, UAT	Test report, defect log
Deployment	Direct, phased, pilot, or parallel	Live system
Maintenance	Corrective, adaptive, perfective, preventive	Patches and new versions

BLACK-BOX TEST
No look at the source. Feed inputs, check outputs. Good for UAT and “does it match the SRS?”

WHITE-BOX TEST
You read the code: branches, loops, conditions. Good for unit tests and finding hidden paths.

DEPLOYMENT STYLES
Direct (big bang) · phased rollout · pilot group · parallel run with the old system still alive.

6.1 Waterfall — finish a phase, then lock it

Work flows downhill. You do not climb back to analysis once design has started. Documentation is heavy. Change is expensive. Use it when requirements are stable, the technology is known, and the project is small enough to plan in advance.

6.2 Agile / Scrum — a working slice every sprint

Work is cut into **sprints** of 2–4 weeks. Each sprint plans, codes, tests, reviews, and ships a usable increment. The **product backlog** is a living list of **user stories** (“As a student, I want…”). Daily standups keep blockers visible. Requirements are allowed to evolve.

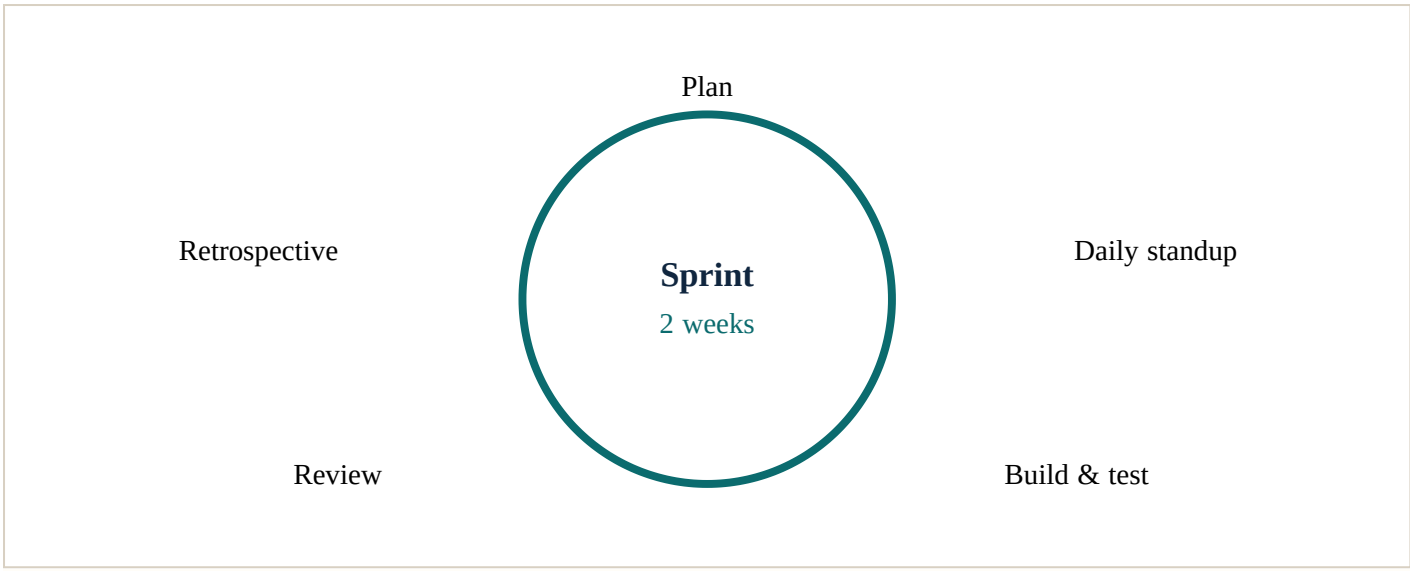


Figure 6. Scrum cycle. The output of every lap is working software plus feedback, not a thicker document.

6.3 Twin case studies — Al-Noor High School

WATERFALL · LIBRARY LMS

1,200 students, 80 teachers, ≈ 2,400 books. Paper register replaced by a desktop LMS: Python 3.10, Tkinter, SQLite, offline Windows. Team of 2 developers + tester + PM, six months. Requirements signed in the SRS and then **frozen**. Tables: STUDENTS, BOOKS, LOANS. Fourteen-day loans and overdue alerts.

Why Waterfall? Needs were clear, deadline fixed, team small, complexity low, documentation required.

AGILE · STUDENT MOBILE APP

Flutter (Android + iOS). 5 sprints × 2 weeks. 3 developers + UI designer + tester. Backlog began with timetable, results, absence SMS, library reserve; bus tracking stayed low priority. Sprint 3 lost SMS and switched to email. Version 1 shipped 5 of 10 stories. Unexplained absences fell about 45%.

Why Agile? Unclear start, many stakeholders, changing mobile platforms, school wanted a demo in weeks not months.

Table 4. Same school, two models — the comparison examiners love

Dimension	Waterfall LMS	Agile mobile app
Requirements	Fixed at the start	Evolving backlog
Client contact	Start and end	Every sprint review
First working build	≈ 6 months	≈ 2 weeks
Change request	Painful / often refused	Parked for the next sprint
Testing	Its own late phase	Continuous inside the sprint
Documentation	Heavy SRS and design packs	Light; working software is the proof

7. How machines talk — OSI and TCP/IP

A **communication model** is a layered set of rules, like a postal system: you write a letter (application), seal an envelope (presentation/session), the post office sorts by city (network), vans move bags (data link), and trucks roll on roads (physical). Nobody on the road needs to read your letter.

7.1 OSI — seven layers (ISO, 1984)

Mnemonic, bottom to top: **Please Do Not Throw Sausage Pizza Away** — Physical, Data link, Network, Transport, Session, Presentation, Application.

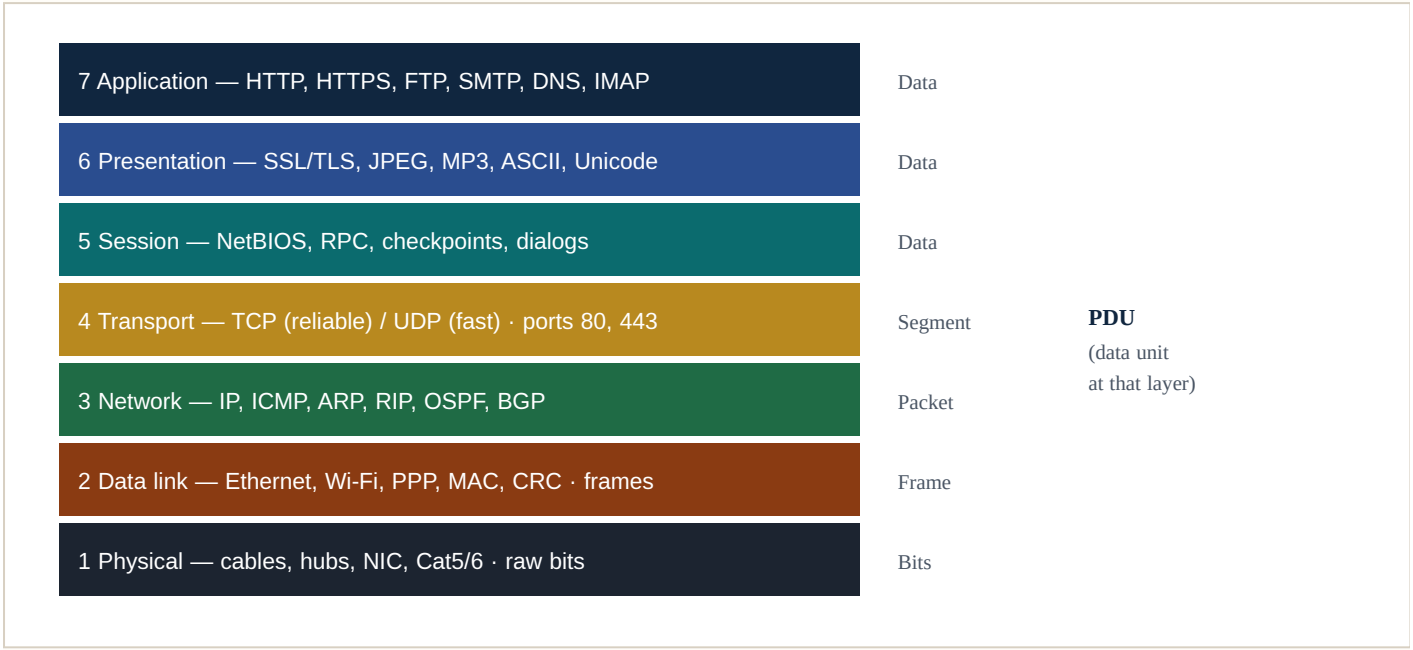


Figure 7. OSI stack and the protocol data unit (PDU) name you must attach to each layer.

Encapsulation (sending): each layer adds its header as the message walks down the stack. **Decapsulation** (receiving): headers are peeled off on the way up. Remember the data-unit chant:

Data → Segment → Packet → Frame → Bits

Table 5. TCP versus UDP (both live at Layer 4)

	TCP	UDP
Connection	Connection-oriented, handshake	Connectionless
Reliability	Acknowledgements, retransmission	Best effort, no retry
Speed	Slower, ordered	Faster, may lose or reorder
Use	Web, email, file transfer	Live video, games, DNS queries

7.2 TCP/IP — the model that actually runs the Internet

Four layers, born in the US Department of Defense in the 1970s. It is an implementation model, not just a teaching diagram.

Table 6. TCP/IP mapped onto OSI

TCP/IP	OSI layers	Example protocols
Application	7 + 6 + 5	HTTP, DNS, FTP, SMTP
Transport	4	TCP, UDP
Internet	3	IPv4, IPv6, ICMP, ARP
Network Access	2 + 1	Ethernet, Wi-Fi, MAC, copper/fibre

Self-check 2 — Systems thinking

1. A school wants a fees system with a law-fixed report format and a six-month deadline. Waterfall or Agile? Why?
2. Which OSI layer adds port numbers? Which adds IP addresses? Which adds MAC addresses?
3. A video call prefers UDP. Give one reason in one sentence.

Answers. (1) Waterfall — requirements are legally frozen and the scope is known. (2) Transport (ports), Network (IP), Data link (MAC). (3) A late packet is useless in a live call; UDP skips retransmission so the picture stays timely.

Recap — pin these twelve lines

1. Discrete / digital = stairs; continuous / analog = ramp.
2. 0 is LOW/FALSE; 1 is HIGH/TRUE.
3. AND needs all 1s; OR needs any 1; NOT flips one bit.
4. $+$ is OR. $1 + 1 = 1$. Precedence: NOT, then AND, then OR.
5. Rows in a truth table $= 2^n$.
6. NAND and NOR are universal gates. XOR is “different”; XNOR is “same”.
7. SOP from minterms ($Y = 1$ rows); POS from maxterms ($Y = 0$ rows).
8. K-maps use Gray code; group 2^n cells; wrap the edges.
9. SDLC: analyse, design, code, test, deploy, maintain.
10. Al-Noor library = Waterfall; Al-Noor app = Agile/Scrum.
11. OSI 7 layers; PDU chant: data, segment, packet, frame, bits.
12. TCP/IP folds OSI 7–5 into Application and 2–1 into Network Access.

Redraw Figure 3 and Figure 7 from memory tonight.